# HireSignal — Project Setup Reference

> Keep this file open alongside your project. Every section tells you **what it is**, **where it lives**, and **how to change it**.

---

## Table of Contents


1. Project Structure
2. Gradle — Build System
3. Docker Infrastructure
4. Service Configuration
5. Auth Service Internals
6. How to Add a New Microservice
7. Common Commands Cheatsheet
8. Ports Reference
9. Credentials Reference
10. Decision Log — Why We Chose X


---

## 1. Project Structure

```
hiresignal/                          ← Git root, always open IntelliJ from here
|
├── build.gradle                     ← Root build: shared plugins + dependencies
for ALL modules
├── settings.gradle                  ← Registers which modules exist (add new
services here)
├── gradlew / gradlew.bat            ← Always use this to build, never system
gradle
|
├── docker-compose.yml               ← All infrastructure (Postgres, Kafka, Redis,
etc.)
|
├── infra/
|   └── postgres/
|       └── init.sql                 ← Runs ONCE on first container start —
creates databases
|
├── common/                          ← Shared library (DTOs, exceptions, utils)
|   └── src/main/java/com/hiresignal/common/
```

```
│       └── dto/auth/                ← Shared request/response objects
│
├── auth-service/                    ← Handles registration, login, JWT
│   ├── build.gradle
│   └── src/main/
│       ├── java/com/hiresignal/auth/
│       │   ├── AuthServiceApplication.java   ← Entry point
│       │   ├── config/SecurityConfig.java    ← Spring Security rules
│       │   ├── controller/AuthController.java
│       │   ├── entity/User.java              ← Maps to users table in auth_db
│       │   ├── entity/Role.java              ← Enum: CANDIDATE, RECRUITER, ADMIN
│       │   ├── repository/UserRepository.java
│       │   ├── security/JwtService.java      ← Token generation + validation
│       │   ├── security/JwtAuthFilter.java   ← Intercepts every request
│       │   └── service/AuthService.java      ← Business logic
│       └── resources/
│           └── application.yml              ← DB url, JWT secret, server port
│
├── job-service/                     ← Job CRUD + Kafka producer
│   ├── build.gradle
│   └── src/main/
│       ├── java/com/hiresignal/job/
│       └── resources/application.yml
│
├── user-service/                    ← Candidate/recruiter profiles
│   ├── build.gradle
│   └── src/main/
│       ├── java/com/hiresignal/user/
│       └── resources/application.yml
│
└── notification-service/            ← Kafka consumer → sends emails/push
    ├── build.gradle
    └── src/main/
        ├── java/com/hiresignal/notification/
        └── resources/application.yml
```

**Rules that never change:**

- Always open IntelliJ from the `hiresignal/` root, never from a service subfolder
- Every service has its own `build.gradle`, its own `application.yml`, its own port
- `common` is a library — it has no `main()` method, it cannot run standalone

---

## 2. Gradle — Build System

### Key files

| File | Purpose | When to edit |
|---|---|---|
| `settings.gradle` | Lists all modules | Adding a new service |
| `build.gradle` (root) | Shared Spring Boot version, Lombok, test deps | Upgrading Spring Boot or adding a dep every service needs |
| `<service>/build.gradle` | Service-specific deps | Adding a dep to one service only |

### Root `build.gradle` — what each section does

```groovy
plugins {
    id 'org.springframework.boot' version '3.2.0' apply false
    // apply false = declared here but NOT applied at root level
    // Each service applies it in its own build.gradle
    // common/ does NOT apply it — it's a library, not a bootable app
}

subprojects {
    // Everything in this block applies to ALL modules automatically
    // Change Java version here: sourceCompatibility = '21'
    // Change group here: group = 'com.hiresignal'
}
```

### `settings.gradle` — registering modules

```groovy
rootProject.name = 'hiresignal'

include 'common'
include 'auth-service'
include 'job-service'
include 'user-service'
include 'notification-service'
// ADD NEW SERVICES HERE — one line per service
```

### How inter-module dependencies work

```groovy
```

```
// In any service's build.gradle:
dependencies {
    implementation project(':common')   // uses our common module
    // 'common' = the folder name, must match what's in settings.gradle
}
```

### Gradle wrapper — always use this

```bash
# Windows
.\gradlew build

# Git Bash / Mac / Linux
./gradlew build

# Build a single module only
./gradlew :auth-service:build

# Run a specific service
./gradlew :auth-service:bootRun

# Skip tests during build (faster iteration)
./gradlew build -x test

# See dependency tree for a module
./gradlew :auth-service:dependencies
```

---

## 3. Docker Infrastructure

### File location: `hiresignal/docker-compose.yml`

### All services and what they do

| Container | Image | Port | Purpose | Data persisted? |
|---|---|---|---|---|
| `postgres` | postgres:16-alpine | 5432 | Primary database for all services | Yes — `postgres-data` volume |
| `redis` | redis:7-alpine | 6379 | Session cache, rate limiting, counters | Yes — `redis-data` volume |

| Container | Image | Port | Purpose | Data persisted? |
|---|---|---|---|---|
| `zookeeper` | confluentinc/cp-zookeeper:7.5.0 | 2181 | Kafka cluster coordinator (required by Kafka) | No |
| `kafka` | confluentinc/cp-kafka:7.5.0 | 9092 | Event streaming between services | Yes — `kafka-data` volume |
| `kafka-ui` | provectuslabs/kafka-ui | 8090 | Visual dashboard for Kafka topics | No |
| `elasticsearch` | elasticsearch:8.11.0 | 9200 | Full-text job search | Yes — `es-data` volume |
| `kibana` | kibana:8.11.0 | 5601 | Log visualization dashboard | No |
| `pgadmin` | dpage/pgadmin4 | 5050 | PostgreSQL visual UI | No |

### Docker networking — critical concept

All containers are on the `hiresignal-net` network.

```
From your laptop  → use localhost:PORT   (e.g. localhost:5432)
From a container  → use container_name:PORT  (e.g. postgres:5432)
```

This is why `application.yml` for local dev uses `localhost`, but if you ever run your Spring Boot services inside Docker too, you must change to container names.

### Common Docker commands

```bash
```

```bash
# Start everything
docker compose up -d

# Start only what you need (Phase 1)
docker compose up -d postgres redis zookeeper kafka kafka-ui

# Start Phase 2 (adds search + logging)
docker compose up -d elasticsearch kibana pgadmin

# Stop everything (data preserved in volumes)
docker compose down

# Stop and DELETE all data (fresh start)
docker compose down -v

# See status of all containers
docker compose ps

# See logs for one container
docker compose logs kafka
docker compose logs -f kafka    # -f = follow (live tail)

# Restart one container
docker compose restart kafka

# Rebuild after changing docker-compose.yml
docker compose up -d --force-recreate kafka
```

**PostgreSQL init script:** `infra/postgres/init.sql`

Runs **once** on first container start. If you need to re-run it:

```bash
docker compose down -v          # deletes volume
docker compose up -d postgres   # starts fresh, runs init.sql again
```

To add a new database for a new service:

```sql
-- Add to infra/postgres/init.sql
CREATE DATABASE newservice_db;
GRANT ALL PRIVILEGES ON DATABASE newservice_db TO hiresignal;
```

### Connecting to PostgreSQL directly

```bash
# List all databases
docker exec -it postgres psql -U hiresignal -d postgres -c "\l"

# Connect to a specific database
docker exec -it postgres psql -U hiresignal -d auth_db

# Inside psql: list tables
\dt

# Inside psql: describe a table
\d users

# Inside psql: quit
\q
```

### Kafka — key concepts

Kafka has two listener configurations. This confuses everyone once.

```yaml
KAFKA_ADVERTISED_LISTENERS:
  PLAINTEXT://localhost:9092        ← your laptop connects here
  PLAINTEXT_INTERNAL://kafka:29092  ← other Docker containers connect here
```

- Your Spring Boot services (running on laptop) use `localhost:9092`
- If a container needs to talk to Kafka, it uses `kafka:29092`

---

## 4. Service Configuration

### `application.yml` location

Each service: `<service-name>/src/main/resources/application.yml`

### What to change and where

### Change database for a service:

```yaml
```

```yaml
# auth-service/src/main/resources/application.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/auth_db   ← change database name here
    username: hiresignal                            ← change if you change docker-co
    password: hiresignal123
```

**Change server port:**

```yaml
server:
  port: 8081   ← must be unique per service
```

**Change JWT token expiry:**

```yaml
# auth-service/src/main/resources/application.yml
jwt:
  secret: <your-secret>
  access-token-expiry: 900000      ← 15 min in ms. Change to 1800000 for 30 min
  refresh-token-expiry: 604800000  ← 7 days in ms
```

**Change Kafka bootstrap server:**

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092   ← change to kafka:29092 if running in Docker
```

**Enable/disable SQL logging:**

```yaml
spring:
  jpa:
    show-sql: true   ← set false in production
```

**Change log level:**

```yaml
```

```yaml
logging:
  level:
    com.hiresignal: DEBUG    ← DEBUG for dev, INFO for production
    org.springframework.security: DEBUG  ← remove in production (too verbose)
```

### JPA ddl-auto values — know these cold

```yaml
spring:
  jpa:
    hibernate:
      ddl-auto: update
```

| Value | What it does | When to use |
|---|---|---|
| `create` | Drops and recreates tables on every start | Never in real use |
| `create-drop` | Creates on start, drops on stop | Unit tests only |
| `update` | Adds missing columns/tables, never drops | Local development |
| `validate` | Checks schema matches entities, crashes if not | Staging/production |
| `none` | Does nothing | Production (use Flyway/Liquibase instead) |

---

## 5. Auth Service Internals

### Request flow

```
POST /api/auth/register
  → AuthController.register()
  → AuthService.register()
      → checks email uniqueness (UserRepository.existsByEmail)
      → BCrypt.encode(password)          strength=12 → ~300ms intentionally slow
      → User.builder()...save()          JPA → INSERT INTO users
      → JwtService.generateAccessToken() HMAC-SHA256 signed, 15 min expiry
      → JwtService.generateRefreshToken() 7 day expiry
  → returns { accessToken, refreshToken, email, role }
```

### JWT structure

A JWT has 3 parts: `header.payload.signature`

```
eyJhbGc...   ←  header  (algorithm: HS256)
eyJzdWI...   ←  payload (sub: email, role: CANDIDATE, exp: timestamp)
SflKxw...    ←  signature (HMAC-SHA256 of header+payload using your secret)
```

Paste any token at jwt.io to inspect it during development.

### Spring Security filter order

Every HTTP request passes through this chain in order:

```
Request → JwtAuthFilter → SecurityFilterChain → Controller
              ↓
    extracts Bearer token
    validates signature
    sets SecurityContext
              ↓
    if no token → passes through (public endpoints handle this)
    if invalid  → SecurityFilterChain returns 403
    if valid    → Controller runs with authenticated user
```

### Adding a new protected endpoint

In `SecurityConfig.java`:

```java
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()    // public
    .requestMatchers("/api/admin/**").hasRole("ADMIN")  // admin only
    .requestMatchers("/api/jobs/**").hasAnyRole("CANDIDATE", "RECRUITER")
    .anyRequest().authenticated()
)
```

### Getting the current user in any controller

```java
@GetMapping("/me")
public ResponseEntity<User> getCurrentUser(Authentication authentication) {
    User user = (User) authentication.getPrincipal();
    return ResponseEntity.ok(user);
}
```

---

## 6. How to Add a New Microservice

Follow every step in order. Do not skip.

### Example: adding `api-gateway` service

### Step 1 — Create folder structure

```bash
# From hiresignal/ root
mkdir -p api-gateway/src/main/java/com/hiresignal/gateway
mkdir -p api-gateway/src/main/resources
mkdir -p api-gateway/src/test/java
```

### Step 2 — Register in `settings.gradle`

```groovy
// hiresignal/settings.gradle
include 'common'
include 'auth-service'
include 'job-service'
include 'user-service'
include 'notification-service'
include 'api-gateway'            // ← add this line
```

### Step 3 — Create `api-gateway/build.gradle`

```groovy
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':common')
    implementation 'org.springframework.boot:spring-boot-starter-web'
    // add service-specific deps here
}
```

### Step 4 — Create the main class

```bash
```

```
cat > api-gateway/src/main/java/com/hiresignal/gateway/ApiGatewayApplication.java <<
package com.hiresignal.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
EOF
```

### Step 5 — Create `application.yml`

```bash
cat > api-gateway/src/main/resources/application.yml << 'EOF'
spring:
  application:
    name: api-gateway

server:
  port: 8080    # pick a unique port — see ports table below

logging:
  level:
    com.hiresignal: DEBUG
EOF
```

### Step 6 — Add database (if service needs one)

Add to `infra/postgres/init.sql`:

```sql
CREATE DATABASE gateway_db;
GRANT ALL PRIVILEGES ON DATABASE gateway_db TO hiresignal;
```

Re-run init script:

```bash
docker compose down -v
docker compose up -d postgres
```

Add datasource to `application.yml`:

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/gateway_db
    username: hiresignal
    password: hiresignal123
  jpa:
    hibernate:
      ddl-auto: update
```

### Step 7 — Verify build

```bash
./gradlew build
# Must show BUILD SUCCESSFUL with new module included
```

### Step 8 — Run the new service

```bash
./gradlew :api-gateway:bootRun
```

---

## 7. Common Commands Cheatsheet

### Build

```bash
./gradlew build                        # build all modules
./gradlew build -x test                # build, skip tests
./gradlew :auth-service:build          # build one module
./gradlew clean build                  # clean then build (fixes weird caching issue
```

### Run services

```bash
./gradlew :auth-service:bootRun        # run auth service
./gradlew :job-service:bootRun         # run job service
# Open separate Git Bash windows for each service
```

### Docker

```bash
docker compose up -d                    # start all containers
docker compose up -d postgres kafka     # start specific containers
docker compose down                     # stop all (data kept)
docker compose down -v                  # stop all + delete data
docker compose ps                       # check status
docker compose logs -f <name>           # follow logs
docker compose restart <name>           # restart one container
```

### PostgreSQL

```bash
# Connect to a database
docker exec -it postgres psql -U hiresignal -d auth_db

# List databases
docker exec -it postgres psql -U hiresignal -d postgres -c "\l"

# List tables in current db
\dt

# See table columns
\d users

# Run a query
SELECT * FROM users LIMIT 10;
```

### Kafka

```bash
```

```bash
# List topics
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list

# Create a topic manually
docker exec -it kafka kafka-topics \
  --bootstrap-server localhost:9092 \
  --create --topic job.posted \
  --partitions 3 --replication-factor 1

# Consume messages from beginning (debug)
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic job.posted \
  --from-beginning
```

### Git

```bash
git init
git add .
git commit -m "feat: initial project setup with multi-module gradle"
# Commit after each working milestone, not after each file
```

---

## 8. Ports Reference

| Port | Service | UI? |
|---|---|---|
| 8081 | auth-service | No — REST API |
| 8082 | job-service | No — REST API |
| 8083 | user-service | No — REST API |
| 8084 | notification-service | No — REST API |
| 8080 | api-gateway (Phase 2) | No — REST API |
| 5432 | PostgreSQL | No — use pgAdmin |
| 6379 | Redis | No — use redis-cli |
| 9092 | Kafka | No — use Kafka UI |
| 2181 | Zookeeper | No |

| Port | Service | UI? |
|---|---|---|
| 8090 | Kafka UI | Yes → localhost:8090 |
| 5050 | pgAdmin | Yes → localhost:5050 |
| 9200 | Elasticsearch | Yes → localhost:9200 (raw JSON) |
| 5601 | Kibana | Yes → localhost:5601 |

**Rule:** Never assign a new service a port already in this table.

---

## 9. Credentials Reference

> These are local development credentials only. Never use these in production.

### PostgreSQL

| Field | Value |
|---|---|
| Host | localhost (from laptop) / postgres (from container) |
| Port | 5432 |
| Username | hiresignal |
| Password | hiresignal123 |
| Databases | auth_db, job_db, user_db |

### Redis

| Field | Value |
|---|---|
| Host | localhost |
| Port | 6379 |
| Password | redis123 |

### pgAdmin

| Field | Value |
|---|---|
| URL | http://localhost:5050 |

| Field | Value |
| --- | --- |
| Email | admin@hiresignal.com |
| Password | admin123 |

**JWT**

| Field | Value | Location to change |
| --- | --- | --- |
| Secret | 404E635266...5970 | auth-service/application.yml → jwt.secret |
| Access token expiry | 900000ms (15 min) | auth-service/application.yml → jwt.access-token-expiry |
| Refresh token expiry | 604800000ms (7 days) | auth-service/application.yml → jwt.refresh-token-expiry |

---

## 10. Decision Log — Why We Chose X

This section is what separates you in interviews. Every architecture decision has a reason.

### Multi-module Gradle over separate repos

One Git repo, one IntelliJ project, one `./gradlew build` command. Shared DTOs in `common` module avoid duplication. Trade-off: harder to deploy services independently (solved later with Docker).

### Database-per-service

`auth_db`, `job_db`, `user_db` are isolated. `job-service` cannot `JOIN` against `auth_db`. Cross-service data access happens only through REST APIs or Kafka events. This enforces service boundaries. Without this discipline, you have a distributed monolith.

### JWT over sessions

Sessions require server-side state — every request must hit Redis or DB to validate. JWT is stateless — any service can validate a token using the shared secret without a network call. Trade-off: tokens cannot be invalidated before expiry (solved with short 15-minute expiry + refresh token rotation).

### BCrypt strength 12

Strength 12 makes each hash take ~300ms on modern hardware. This makes brute-force attacks computationally infeasible — an attacker can only try ~3 passwords/second. Trade-off: registration/login is intentionally slow, which is acceptable.

### Kafka over direct REST calls between services

When a job is posted, three things need to happen: index in Elasticsearch, send notifications, run AI matching. With REST, `job-service` must call all three synchronously — tight coupling, cascading failures. With Kafka, `job-service` publishes one event. Each consumer handles it independently, at its own pace. Trade-off: eventual consistency (notifications arrive milliseconds later, not immediately).

### Redis for caching

Database queries are expensive. Job listings that 1000 users request in a second should be cached. Redis stores the result in memory with a TTL. When the cache expires, one request hits the DB and refreshes it. Trade-off: stale data for TTL duration (acceptable for job listings, not acceptable for account balances).

### `ddl-auto: update` in development

Hibernate automatically creates/updates tables based on your `@Entity` classes. Zero manual SQL during development. Trade-off: dangerous in production (can lock tables during migrations). Production uses `validate` with Flyway for controlled migrations.